

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT (Data Structures & Algorithms)

L/O/G/O

GV: HUỖNH THỊ THANH THƯỜNG

Email: hh.thanhthuong@gmail.com

thuonghtt@uit.edu.vn

CẤU TRÚC DỮ LIỆU ĐỘNG

C H Ứ Ơ N G 3

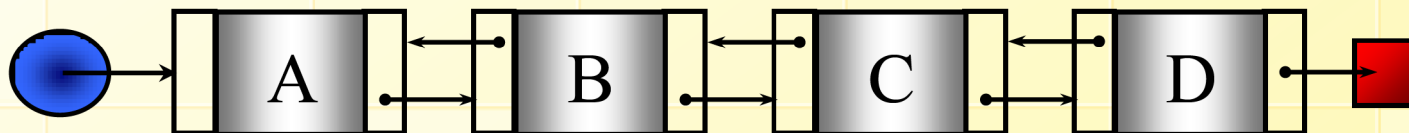


L/O/G/O

www.themegallery.com

DANH SÁCH LIÊN KẾT ĐÔI

- ✓ Mỗi phần tử trong danh sách có kết nối với 1 phần tử đứng trước và 1 phần tử đứng sau nó.
- *pPrev*: con trỏ liên kết với phần tử đứng trước
- *pNext*: con trỏ liên kết với phần tử đứng sau



DANH SÁCH LIÊN KẾT ĐÔI

(Cài đặt bằng con trỏ)

```
typedef struct DNode
{
    <kiểu dữ liệu> info;
    DNode* pPrev;
    DNode* pNext;
};
typedef struct DList
{
    int n;
    DNode* pHead;
    DNode* pTail;
};
```


TẠO MỘT PHẦN TỬ

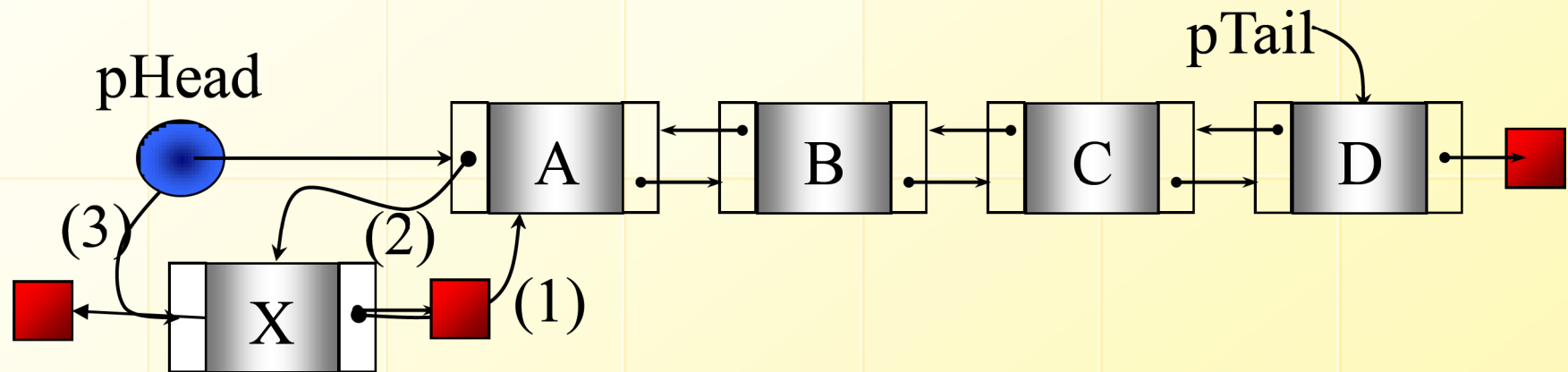
- ✓ Thủ tục GetNode để tạo ra một phần tử cho danh sách với thông tin chứa trong x

```
DNode* GetNode(<kiểu dữ liệu> x)
{
    DNode *p;
    p = new DNode; // Cấp phát vùng nhớ cho phần tử
    if ( p==NULL) return NULL;
    p->info = x;    // Gán thông tin cho phần tử p
    p->pPrev = p->pNext = NULL;
    return p;
}
```

THÊM MỘT PHẦN TỬ

- ✓ 4 vị trí thêm phần tử mới vào danh sách:
 - Thêm vào đầu danh sách
 - Thêm vào cuối danh sách
 - Chèn vào danh sách sau một phần tử q
 - Chèn vào danh sách trước một phần tử q

THÊM MỘT PHẦN TỬ VÀO ĐẦU LIST



THÊM MỘT PHẦN TỬ VÀO ĐẦU LIST

✓ Cài đặt:

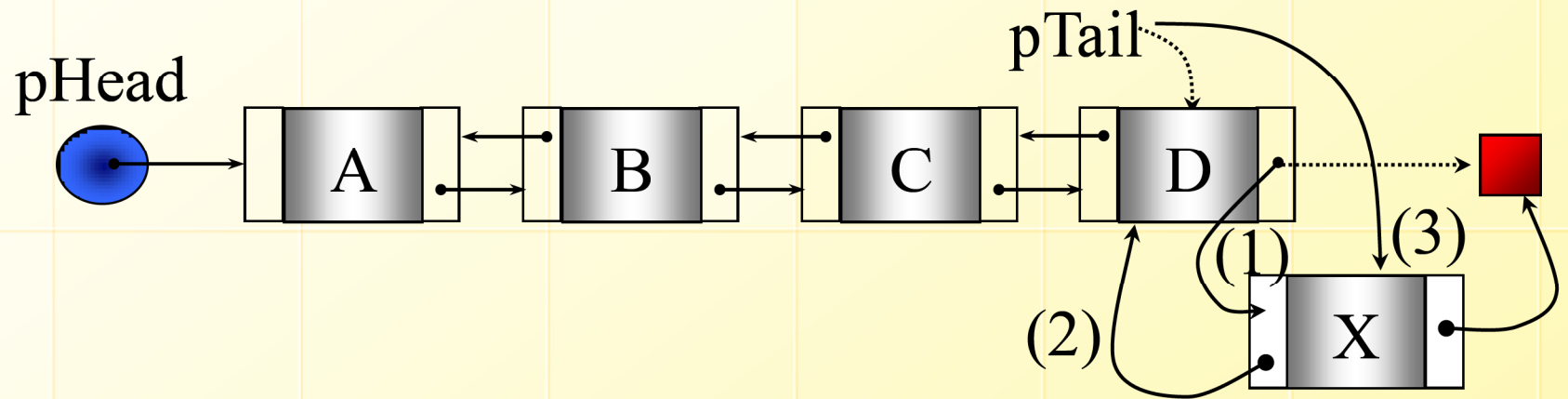
```
void AddFirst (DList &L, DNode* new_ele)
{
    if (L.pHead==NULL)
    {
        L.pHead = new_ele;
        L.pTail = L.pHead;    }
    else
    {
        new_ele->pNext = L.pHead; // (1)
        L.pHead->pPrev = new_ele; // (2)
        L.pHead = new_ele;        // (3)    }
}
```

THÊM MỘT PHẦN TỬ DỮ LIỆU VÀO ĐẦU

✓ Cài đặt:

```
DNode* InsertHead(DList &L, <kiểu dữ liệu> x)
{
    DNode* new_ele = GetNode(x);
    if (new_ele == NULL) return NULL;
    AddFirst(L, new_ele)
    return new_ele;
}
```

THÊM MỘT PHẦN TỬ VÀO CUỐI LIST



THÊM MỘT PHẦN TỬ VÀO CUỐI LIST

✓ Cài đặt:

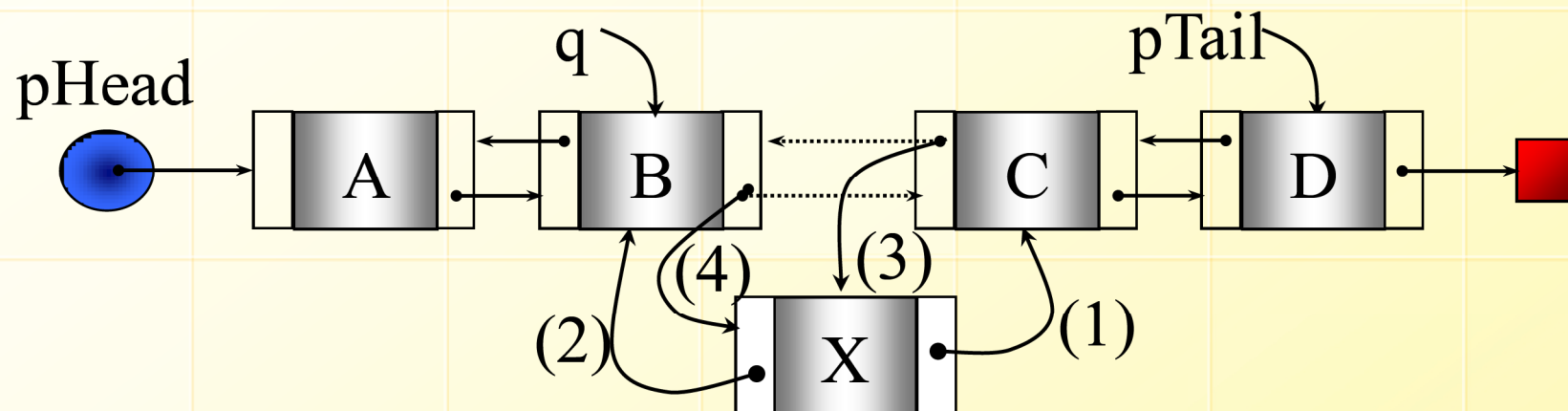
```
void AddTail(DList &L, DNode *new_ele)
{
    if (L.pHead==NULL)
    {
        L.pHead = new_ele;
        L.pTail = L.pHead;    }
    else
    {
        L.pTail-> pNext = new_ele;    // (1)
        new_ele->pPrev = L.pTail;    // (2)
        L.pTail = new_ele;            // (3)    }
}
```

THÊM MỘT PHẦN TỬ DỮ LIỆU VÀO CUỐI

✓ Cài đặt:

```
DNode* InsertTail(DList &L, <kiểu dữ liệu> x)
{
    DNode* new_ele = GetNode(x);
    if (new_ele == NULL) return NULL;
    AddTail(L, new_ele)
    return new_ele;
}
```


CHÈN MỘT PHẦN TỬ SAU q



CHÈN MỘT PHẦN TỬ SAU q

✓ Cài đặt:

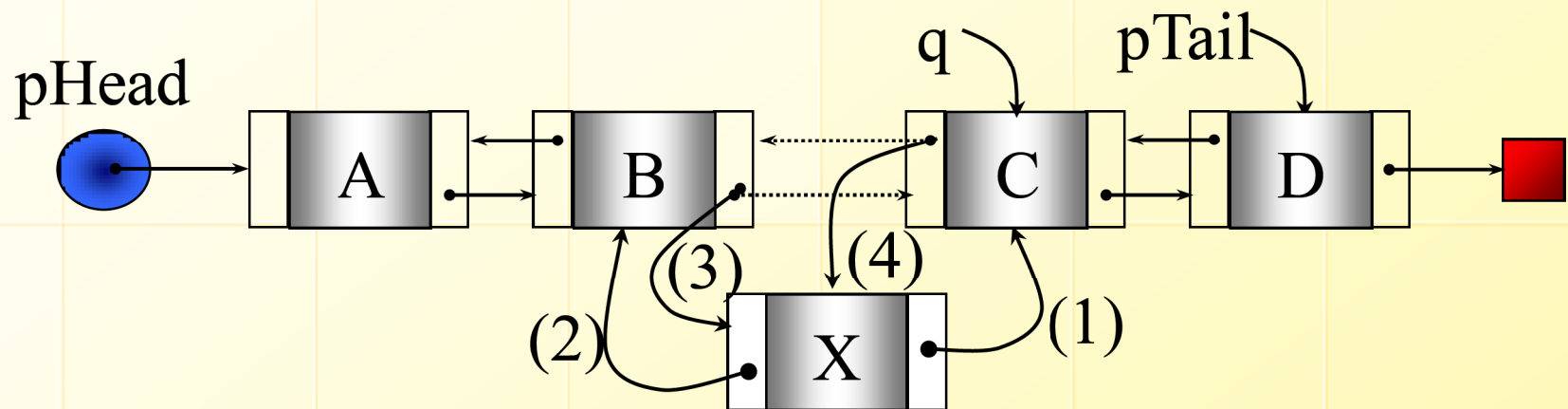
```
void AddAfter(DList &L, DNode q, DNode* new_ele)
{
    DNode* p = q->pNext;
    if ( q!=NULL)
    {
        new_ele->pNext = p;           //(1)
        new_ele->pPrev = q;           //(2)
        if(p != NULL)
            p->pPrev = new_ele;       //(3)
        q->pNext = new_ele;           //(4)
        if(q == L.pTail)
            L.pTail = new_ele;    }
    else //chèn vào đầu danh sách
        AddFirst(L, new_ele);
}
```

THÊM MỘT PHẦN TỬ DỮ LIỆU SAU q

✓ Cài đặt:

```
void InsertAfter(DList &L, DNode q, <kiểu dữ liệu> x)
{
    DNode* new_ele = GetNode(x);
    if (new_ele == NULL)
        return NULL;
    AddAfter(L, q, new_ele)
}
```

CHÈN MỘT PHẦN TỬ TRƯỚC q



CHÈN MỘT PHẦN TỬ TRƯỚC q

✓ Cài đặt:

```
void AddBefore(DList &L, DNode q, DNode* new_ele)
{
    DNode* p = q->pPrev;
    if ( q!=NULL)
    {
        new_ele->pNext = q;           //(1)
        new_ele->pPrev = p;           //(2)
        q->pPrev = new_ele;           //(3)
        if(p != NULL)
            p->pNext = new_ele;       //(4)
        if(q == L.pHead)
            L.pHead = new_ele;    }
    else //chèn vào đầu danh sách
        AddTail(L, new_ele);
}
```

THÊM MỘT PHẦN TỬ DỮ LIỆU TRƯỚC q

✓ Cài đặt:

```
void InsertBefore (DList &l, DNode q, <kiểu dữ liệu> x)
{
    DNode* new_ele = GetNode(x);
    if (new_ele == NULL) return NULL;
    DNode* p = q->pPrev;
    AddBefore(L, q, new_ele)
}
```

HỦY MỘT PHẦN TỬ KHỎI DSLK ĐÔI

- Hủy phần tử đầu danh sách
- Hủy phần tử cuối danh sách
- Hủy một phần tử đứng sau phần tử q
- Hủy một phần tử đứng trước phần tử q
- Hủy 1 phần tử có khóa k

HỦY PHẦN TỬ ĐẦU DANH SÁCH

```
<kiểu dữ liệu> RemoveHead (DList &L)
{
    DNode *p;
    <kiểu dữ liệu> x;
    if ( L.pHead != NULL)
    {
        p = L.pHead;
        x = p->info;
        L.pHead = L.pHead->pNext;
        L.pHead->pPrev = NULL;
        delete p;
        if(L.pHead == NULL)
            L.pTail = NULL;
        else
            L.pHead->pPrev = NULL;
    }
    return x;
}
```


HỦY PHẦN TỬ CUỐI DANH SÁCH

```
<kiểu dữ liệu> RemoveTail (DList &L)
{
    DNode *p;
    <kiểu dữ liệu> x = NULLDATA;
    if ( L.pTail != NULL)
    {
        p = L.pTail;
        x = p->info;
        L.pTail = L.pTail->pPrev;
        L.pTail->pNext = NULL;
        delete p;
        if(L.pHead == NULL)
            L.pTail = NULL;
        else L.pHead->pPrev = NULL;
    }
    return x;
}
```

HỦY PHẦN TỬ ĐỨNG SAU q

```
void RemoveAfter (DList &L, DNode *q)
{
    DNode *p;
    if ( q != NULL)
    {
        p = q ->pNext ;
        if ( p != NULL)
        {
            q->pNext = p->pNext;
            if(p == L.pTail)
                L.pTail = q;
            else
                p->pNext->pPrev = q;
            delete p;
        }
    }
    else
        RemoveHead(L);
}
```

HỦY PHẦN TỬ ĐỨNG TRƯỚC q

```
void RemoveBefore (DList &L, DNode *q)
{
    DNode *p;
    if ( q != NULL)
    {
        p = q ->pPrev;
        if ( p != NULL)
        {
            q->pPrev = p->pPrev;
            if(p == L.pHead)
                L.pHead = q;
            else
                p->pPrev->pNext = q;
            delete p;
        }
    }
    else
        RemoveTail(L);
}
```

HỦY PHẦN TỬ CÓ KHÓA k

```
int RemoveNode(DList &L, <kiểu dữ liệu> k)
{
    DNode *p = L.pHead;          DNode *q;
    while( p != NULL)
    {
        if(p->info == k) break;
        p = p->pNext;             }
    if(p == NULL) return 0; //Không tìm thấy k
    q = p->pPrev;
    if ( q != NULL)
    {
        p = q ->pNext ;
        if ( p != NULL)
        {
            q->pNext = p->pNext;
            if(p == L.pTail) L.pTail = q;
            else p->pNext->pPrev = q; }
    }
}
```

HỦY PHẦN TỬ CÓ KHÓA k

```
int RemoveNode(DList &L, <kiểu dữ liệu> k)
{
    DNode *p = L.pHead;          DNode *q;
    while( p != NULL)
    {
        if(p->info == k) break;
        p = p->pNext;
    }
    if(p == NULL) return 0; //Không tìm thấy k
    q = p->pPrev;
    if ( q != NULL)
    {
        p = q ->pNext ;
        if ( p != NULL)
        {
            q->pNext = p->pNext;
            if(p == L.pTail) L.pTail = q;
            else p->pNext->pPrev = q;
        }
    }
}
```

HỦY PHẦN TỬ CÓ KHÓA k

```
int RemoveNode(DList &L, <kiểu dữ liệu> k)
{
    -----
    else //p là phần tử đầu xâu
    {
        L.pHead = p->pNext;
        if(L.pHead == NULL)
            L.pTail = NULL;
        else L.pHead->pPrev = NULL;    }
    delete p;
    return 1;
}
```

SẮP XẾP DANH SÁCH LIÊN KẾT ĐÔI

- Về cơ bản không khác gì sắp xếp trên DSLK đơn.
- Lưu ý: cần bảo toàn các mối liên kết hai chiều trong khi sắp xếp.

SẮP XẾP DANH SÁCH LIÊN KẾT ĐÔI

```
void QuickSort(DList & L)
{
    DNode *p, *X;           // X chỉ đến phần tử cần canh
    DList L1, L2;
    if(L.pHead == L.pTail)   return; //đã có thứ tự
    L1.pHead == L1.pTail = NULL; //khởi tạo
    L2.pHead == L2.pTail = NULL;
    X = L.pHead; L.pHead = X->pNext;
    while(L.pHead != NULL)   //Tách L thành L1, L2
    {
        p = L.pHead;
        L.pHead = p->pNext; p->pNext = NULL;
        if (p->info <= X->info) AddTail(L1, p);
        else                    AddTail(L2, p);
    }
}
```


SẮP XẾP DANH SÁCH LIÊN KẾT ĐÔI

```
void QuickSort(DList & L)
{
    .....
    QuickSort(L1);           //Gọi đệ qui để sort L1
    QuickSort(L2);           //Gọi đệ qui để sort L2
    //Nối L1, X và L2 lại thành L đã sắp xếp
    if(L1.pHead != NULL)
    {
        L.pHead = L1.pHead; L1.pTail->pNext = X;
        X->pPrev = L1.pTail;
    }
    else    L.pHead = X;
    X->pNext = L2;
    if(L2.pHead != NULL)
    {
        L.pTail = L2.pTail;
        L2->pHead->pPrev = X;
    }
    else    L.pTail = X;
}
```

SO SÁNH

	DANH SÁCH LIÊN KẾT ĐƠN	DANH SÁCH LIÊN KẾT ĐÔI
ƯU		- Từ 1 phần tử bất kỳ có thể truy xuất 1 phần tử bất kỳ khác. - Dễ dàng hủy phần tử DSLK đôi
KHUYẾT	- Chỉ có thể truy xuất đến các phần tử đứng sau 1 phần tử cho trước. - Thao tác hủy phần tử tốn chi phí $O(n)$.	- Tốn chi phí gấp đôi so với DSLK đơn cho việc lưu trữ các mối liên kết. - Việc cập nhật nặng nề hơn trong một số trường hợp

DANH SÁCH LIÊN KẾT CÓ THỨ TỰ (ORDERED LIST)

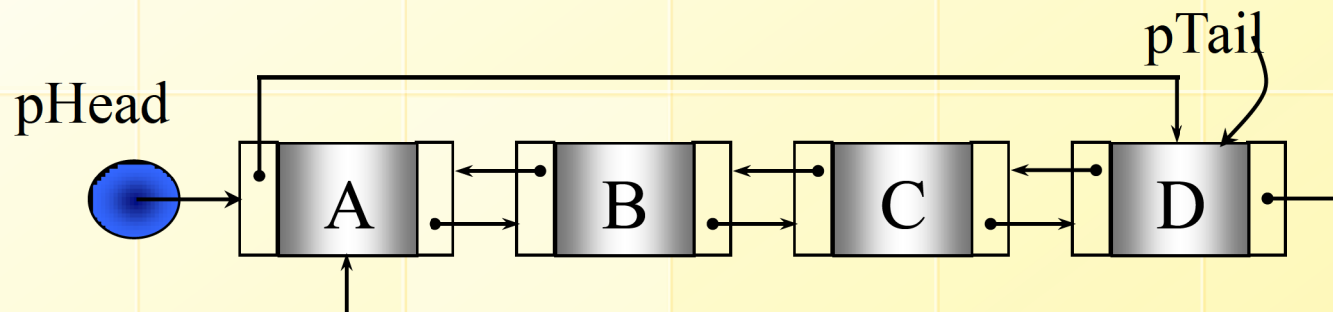
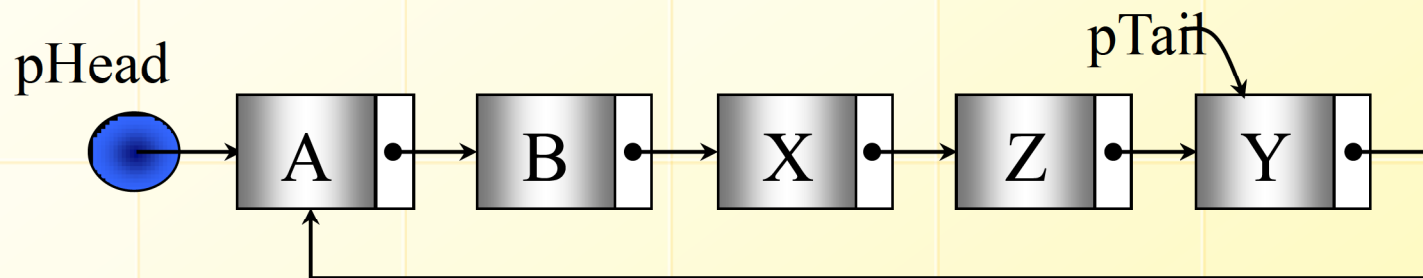
- ✓ Chứa các đối tượng theo một trình tự nhất định (theo 1 khóa sắp xếp nào đó).
- ✓ Việc thêm phần tử vào danh sách phải bảo toàn thứ tự này.
- ✓ Có thể cài đặt bằng DSLK đơn hoặc DSLK đôi với việc định nghĩa lại duy nhất 1 phép thêm phần tử.

THÊM 1 PHẦN TỬ

```
node* InsertNode(List & L, <kiểu dữ liệu> X)
{
    node* q = NULL, *p = L.pHead;
    if(IsEmpty(L)) return InsertHead(L, X);
    while(p)
    {
        if(p->info >= X)      break;
        q = p;
        p = q->pNext;
    }
    node * pT = new node;
    pT->info = X; pT->pNext= p;
    if(q)      q->pNext      = pT;
    else      L.pHead      = pT;
    if(q == L->pTail)      L.pTail = pT;
    return pT;
}
```

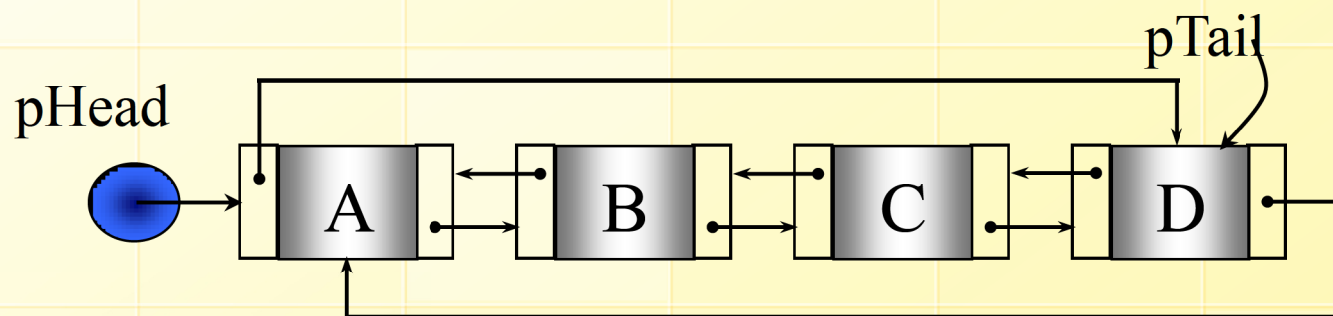
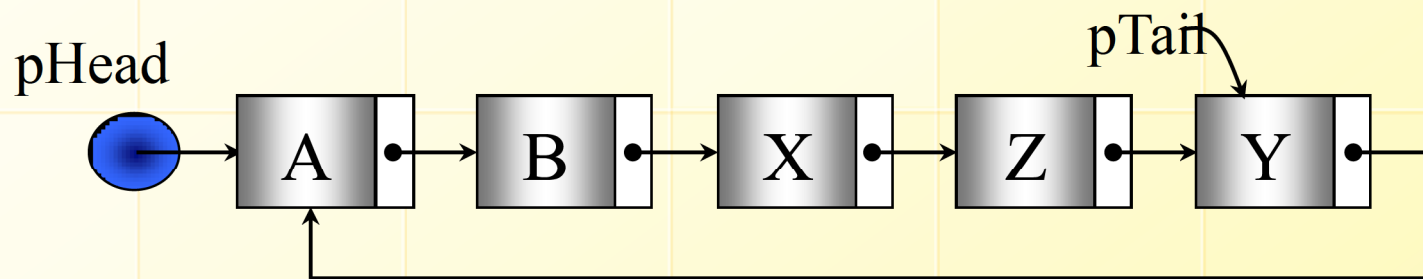
DANH SÁCH LIÊN KẾT VÒNG

- ✓ Là 1 DSLK đơn (hoặc đôi) mà **phần tử cuối** thay vì mang giá trị NULL, **trở tới phần tử đầu** danh sách.
- ✓ Có thể xuất phát từ 1 phần tử bất kỳ để duyệt toàn bộ danh sách.



DANH SÁCH LIÊN KẾT VÒNG

- ✓ Không có phần tử đầu rõ rệt → có thể đánh dấu 1 phần tử bất kỳ xem như phần tử đầu để kiểm tra việc duyệt đã qua hết các phần tử của DS hay chưa.



TÌM 1 PHẦN TỬ TRÊN DSLK VÒNG

```
node* Search(List &L, <kiểu dữ liệu> x)
{
    node *p;
    p = L.pHead;
    do
    {
        if ( p->info == x) return p;
        p = p->pNext;
    } while (p != L.pHead); // chưa đi giáp vòng
    return p;
}
```

THÊM MỘT PHẦN TỬ VÀO ĐẦU LIST

✓ Cài đặt:

```
void AddHead(List &L, node *new_ele)
{
    if(L.pHead == NULL)
    {
        L.pHead = L.pTail = new_ele;
        L.pTail->pNext = L.pHead;
    }
    else
    {
        new_ele->pNext = L.pHead;
        L.pTail->pNext = new_ele;
        L.pHead = new_ele;
    }
}
```


THÊM MỘT PHẦN TỬ VÀO CUỐI LIST

✓ Cài đặt:

```
void AddTail(LIST &L, NODE *new_ele)
{
    if(L.pHead == NULL)
    {
        L.pHead = L.pTail = new_ele;
        L.pTail->pNext = L.pHead;
    }
    else
    {
        new_ele->pNext = L.pHead;
        L.pTail->pNext = new_ele;
        L.pTail = new_ele;
    }
}
```

CHÈN MỘT PHẦN TỬ SAU q

✓ Cài đặt:

```
void AddAfter(LIST &L, NODE *q, NODE *new_ele)
{
    if(L.pHead == NULL)
    {
        L.pHead = L.pTail = new_ele;
        L.pTail->pNext = L.pHead;
    }
    else
    {
        new_ele->pNext = q->pNext;
        q->pNext = new_ele;
        if(q == L.pTail)
            L.pTail = new_ele;
    }
}
```

HỦY PHẦN TỬ ĐẦU DANH SÁCH

```
void RemoveHead(List &L)
{
    node *p = L.pHead;
    if(p == NULL) return;
    if (L.pHead == L.pTail)
        L.pHead = L.pTail = NULL;
    else
    {
        L.pHead = p->pNext;
        if(p == L.pTail)
            L.pTail->pNext = L.pHead;
    }
    delete p;
}
```

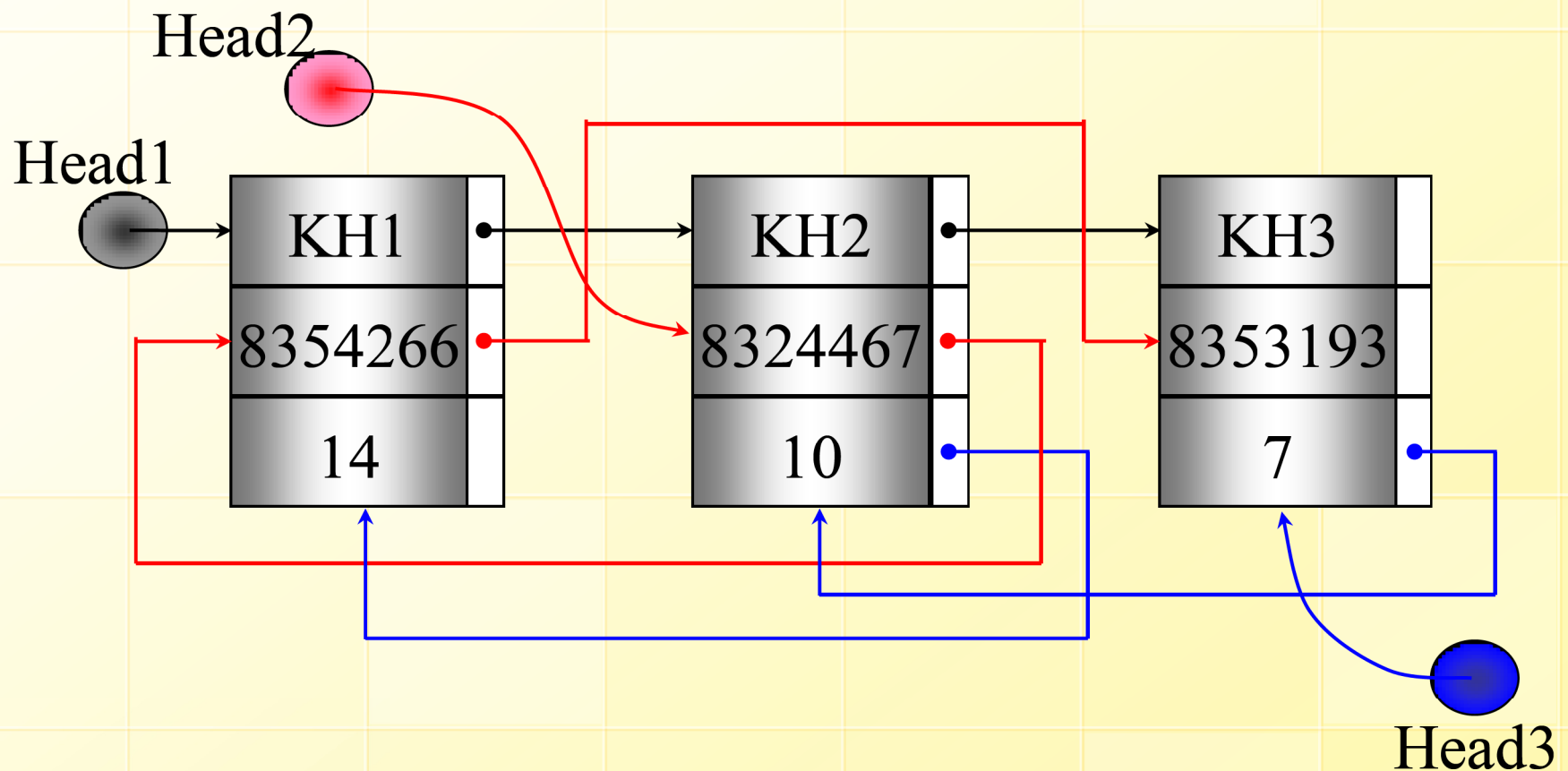
HỦY PHẦN TỬ SAU q

```
void RemoveAfter(List &L, node *q)
{
    node *p;
    if(q != NULL)
    {
        p = q ->pNext ;
        if ( p == q)
            L.pHead = L.pTail = NULL;
        else
        {
            q->pNext = p->pNext;
            if(p == L.pTail)
                L.pTail = q;
        }
        delete p;
    }
}
```

DANH SÁCH CÓ NHIỀU MỐI LIÊN KẾT

- ✓ Mỗi phần tử có nhiều khóa và được liên kết với nhau theo từng loại khoá.
- ✓ Sử dụng trong các ứng dụng quản lý một cơ sở dữ liệu lớn với những nhu cầu tìm kiếm dữ liệu theo những khoá khác nhau.
- ✓ Ví dụ: Quản lý danh mục điện thoại thuận tiện cho việc in danh mục theo những trình tự khác nhau :
 - tên khách tăng dần,
 - theo số điện thoại tăng dần,
 - thời gian lắp đặt giảm dần.

DANH SÁCH CÓ NHIỀU MỐI LIÊN KẾT



DANH SÁCH TỔNG QUÁT

- ✓ Mỗi phần tử của danh sách có thể lại là một danh sách khác.
- ✓ Các thao tác trên một danh sách tổng quát được xây dựng dựa trên cơ sở các thao tác trên danh sách liên kết.

The background features a light pink grid on the left side. Three realistic water droplets are positioned in the upper-middle section of the grid. On the right side, there are large, overlapping, semi-circular shapes in blue, red, and yellow, separated by white borders. A small grey shape is visible in the top right corner.

Thank You!

L/O/G/O